INSTALILY CASE STUDY · MAY 2026

PartSelect Chat Agent

A production-grade customer-support agent for refrigerator and dishwasher parts. Five tools. Structured KG. Selective validator.

CURRENT RESULTS

17/17
EVAL CASES PASS

5
TYPED TOOLS

Prepared by **Sarthak Chandarana**
sarthak.chandarana99@gmail.com
github.com/sarthakc123/instalily-partselect-agent

01 THE PROBLEM

Appliance parts support is slow, expensive, and brittle.

Why it matters

- Live agent contacts cost **$5–$8 each**. Most are repeat questions: *"will this part fit my model?"*
- **Compatibility uncertainty** is the top driver of cart abandonment on parts e-commerce.
- Returns from wrong-part purchases compound the cost with shipping, restocking, and brand erosion.

Four question patterns that must be answered correctly ***and explain themselves***

Install	*How do I install PS11752778?*
Compatibility	*Does this part fit my model?*
Troubleshoot	*My ice maker is broken, what part?*
Compound	*Symptom + model + install in one turn.*

02 THE REAL TEST

If the compound query works, the others fall out for free.

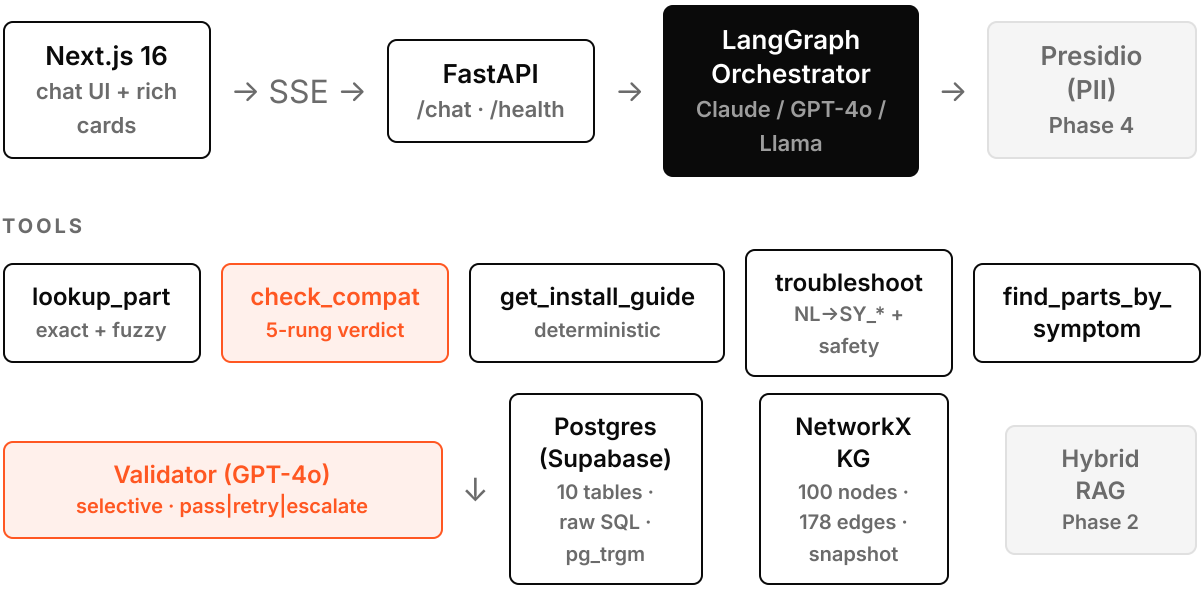
> "Ice maker on my Whirlpool WRF555SDFZ is broken — what part do I need, and how do I install it?"

One message. Four tool calls in a single turn:

01	`troubleshoot`	Natural language → canonical symptom → ranked candidates with likelihood scores
02	`find_parts_by_symptom`	KG traversal (symptom→part→model), annotated with model fitment
03	`check_compatibility`	Structured edge verdict — never LLM reasoning over prose
04	`get_install_guide`	Deterministic step-by-step filtered by part_id

03 ARCHITECTURE

One orchestrator. Five typed tools. Selective validator.



04 DESIGN CHOICE 01

Orchestrator + typed tools, not router-to-specialists.

Why this shape

- Every action is a typed Python function with a JSON Schema contract. The entire action surface is five entries in `tools/registry.py`.
- No tool calls an LLM internally. Results are auditable, fast, and regression-proof.
- Single conversational thread means session memory (`last_part` , `brand` , `model`) carries across turns without specialist coordination.

Tradeoff named

Less *agentic*. We do not get emergent multi-agent behavior.

For this domain that is the right trade: the failure cost of a hallucinated compatibility verdict outweighs the upside of a more creative agent.

> "compatibility verdicts come from edges; prose inference is a marked, lower-confidence fallback."

05 DESIGN CHOICE 02

Knowledge graph as a structured backbone. Not GraphRAG.

The verdict ladder

1	Missing entity	unknown · low
2	KG edge exists	yes · high
3	Cross-appliance types differ	no · high
4	Install-guide series hint matches	inferred · medium
5	No edge otherwise	no · medium

Rung 4 triggers the validator. Rung 3 fires on the case-study "trick" query.

The trick question — live in the eval

"Is PS11752778 compatible with my WDT780SAEM1?"

PS11752778 is a refrigerator part. WDT780SAEM1 is a dishwasher.

> "PS11752778 is not compatible with your WDT780SAEM1, and the reason is an **appliance type mismatch**. PS11752778 is a refrigerator ice maker assembly. WDT780SAEM1 is a dishwasher."

Most agents say "no fitment entry" and leave the reason unstated.

06 DESIGN CHOICE 03

Six hard rules. Enforced in code, verified in the eval.

01	**No em dashes in any user-facing text.**	Eval asserts U+2014 absent in every reply.
02	**PII never enters the LLM context window.**	Presidio inbound · ticket form bypass · log redaction.
03	**Compatibility verdicts from structured edges.**	Verdict ladder; prose inference marked + validator-gated.
04	**Safety symptoms short-circuit to escalation.**	Regex pattern match pre-empts LLM and KG.
05	**Fuzzy SKU matches require user confirmation.**	Repo returns `list[Part]`, never `Part`.
06	**Validator never silently passes a failure.**	`pass | retry | escalate`, retry cap of 1.

> These are the failure modes that get a vendor **fired**. Getting them right is what unblocks enterprise sales.

07 DESIGN CHOICE 04

Provider-agnostic LLM gateway. Insurance, not lock-in.

One normalized event union

```
TextDelta
ToolCallStart · ToolCallDelta · ToolCallComplete
Usage · Done · StreamError
```

Same orchestrator code runs on Anthropic, OpenAI, and Groq. Per-request `X-LLM-Provider` header flips providers mid-conversation.

Insurance benefits

- **Rate-limit resilience.** Groq 429? Switch to Anthropic; user never sees it.
- **Cross-family validator.** Validator uses a different LLM family from the orchestrator. Diversity catches errors one family makes that another does not.
- **Cost flexibility.** Cheap Groq for utility role; premium Claude for orchestrator.
- **No vendor lock-in.** A new provider is one file.

08 EVAL RESULTS

17 / 17 on the case-study eval suite.

CATEGORY	CASES	RESULT
Install	2	PASS
Compatibility (all 5 verdict rungs)	5	PASS
Troubleshoot	1	PASS
Compound — the real test	1	PASS
Edge (fuzzy, case, missing, safety)	4	PASS
Out-of-scope	2	PASS
Adversarial (prompt injection)	1	PASS
Validator (selective trigger)	1	PASS

17/17

191s wall clock

Methodology

Deterministic checks over tool payloads and final text — not LLM-as-judge. A failed `p.verdict == 'yes'` tells you exactly which row in the ladder broke. Every reply printed verbatim in `docs/eval_results.md`.

09 BUSINESS IMPACT

Three categories of value, all unlocked by the same POC.

DEFLECTION

~$1.8M

illustrative annual cost reduction at 50k contacts/month × 60% deflection × $6 saved per contact.

CONVERSION LIFT

A confident, **explained** yes / no / inferred verdict directly recovers cart abandonment from compatibility uncertainty — the largest single driver on parts sites.

ENTERPRISE-SALES UNLOCK

> PII never leaves the boundary. Safety symptoms always escalate. These are the failure modes that get a vendor **removed from a procurement shortlist**.

TIME TO VALUE

50 parts seeded synthetically. A real catalog ingest swaps **the data, not the architecture**. Phase 2 is scrape-and-load, not rebuild.

10 SCALING ROADMAP

Architecture has explicit seams. Each phase is a drop-in.

PHASE	WHAT	HOW IT SLOTS IN
02	**Real PartSelect scrape + hybrid RAG (BM25 + dense + RRF + rerank) over repair stories.**	`VectorStore` protocol → Chroma → pgvector swap. Scraper hits same Postgres schema.
03	**Validator expansion — every troubleshoot and inferred-compat path now graded.**	One conditional edge in `graph.py`. Validator already built.
04	**Production hardening: rate limits, provider fallback chain, Neo4j swap, LangSmith, PagerDuty.**	`KnowledgeGraph` protocol → Neo4j is one file. FastAPI is stateless; KG snapshot loads in <10ms.
05	**Hosted demo on Vercel (frontend) + Fly.io (backend) + Supabase pooler.**	Already env-driven. `NEXT_PUBLIC_BACKEND_URL` + `CORS_ORIGINS`.

THANKS FOR READING

Happy to walk through the code live.

SOURCE + RUN INSTRUCTIONS + EVAL REPORT

github.com/sarthakc123/instalily-partselect-agent

17/17 EVAL

Every assistant reply verbatim in `docs/eval_results.md`.

QUICKS

`pip instal` `.` + on Supaba project Groq Al Runs anywhe

Sarthak Chandarana · **sarthak.chandarana99@gmail.com**